

Incident report — Trade copies failed: broker overload, order timeouts and a retry loop that never stopped

2026-10-05 (detected) → 2026-10-06 (fixed on staging) · Trading worker / trade execution

Severity	High	Status	Fixed — staging deployed 2026-10-06 06:37:21 UTC; production pending
Affected user(s)	36 customer accounts rate-limited; 1 trade copy failed; 1 signal opened three positions	Component	Trading worker — order placement, account management, partial take-profit handling
Root cause	Two jobs sent broker requests hundreds of times per second, overwhelming the connection; our timeout (20 s) was shorter than the bridge's (60 s)	Data impact	Event log table grew to 4.42 GB (same day, separate cause)

1. Executive summary (plain English)

On 5 October customers started reporting that trades were not being copied. Two different jobs inside the worker were each asking the broker for information hundreds of times per second. The connection that carries those requests — the bridge — became saturated. When a genuine trade order then went out, it could not get an answer within 20 seconds, the worker gave up waiting, and the order was recorded as "outcome unknown" even though it may well have been placed. One trade copy failed permanently as a result, and during the busiest hour **36 different customer accounts** were answered with "too many requests".

Two further problems surfaced while we were investigating. The event-log table had grown to 4.42 gigabytes because nothing ever deleted from it, and one take-profit signal opened **three** positions where the system only recorded one, leaving 0.04 lots running for up to seven minutes with no trade record.

What we changed, in plain terms:

- The two jobs now **wait progressively longer between attempts** instead of hammering the broker many times a second. A job that keeps failing with the same unfixable error waits up to 30 minutes between attempts, and parks for six hours after twelve failed attempts.
- An order that times out is now **checked with the broker and either adopted or retried** instead of being left permanently uncertain.
- Requests are **spread evenly** across both available bridge channels instead of arriving in bursts.
- The event-log table is now **pruned daily** (4.42 GB down to 477 MB).
- A stuck take-profit task can only be **closed off after four independent pieces of evidence** that the position really has gone — never on a guess, and never while the position is still open.

None of these changes closes, cancels or duplicates a live position. Every check fails towards "wait and do nothing", never towards "act".

The fixes passed **four rounds of code review** and the **complete worker test suite (2,210 of 2,210 tests passing)**, and were deployed to **staging on 6 October at 06:37 UTC**. In the 83 minutes after that deployment there were no rate-limit errors and no failure spam at all. **Production is still running the old code** (last deployed 4 October) — moving it forward is a separate, deliberate decision.

2. Issue encountered — and what fixed it

All changes below are on branch `fix/mtapi-rate-limit` and were reviewed four times before sign-off.

a) The order that could not be confirmed. At 15:29:08 UTC the worker placed a pending buy order (BuyStop, XAUUSD, 0.06 lots) for one customer's demo account. The broker did not answer within 20 seconds, so the worker stopped waiting. Because an order that times out might still have been accepted, the system correctly refused to guess and recorded the outcome as unknown. Two alerts fired for this one event — one for the timeout, one saying the copy had failed — because the same event is reported from two places.

The errors as they appeared:

- Sentry: broker_order_ambiguous:** Broker OrderSend timed out; outcome requires reconciliation — one event on 5 October, and it belongs to this trade.
- Railway: 15:28:47 [mtapiProvider] OrderSendSafe retry id=167f8c47... attempt=2/3: MTAPI OrderSendSafe timed out**, and then **15:29:08 [tradeExecutor] OrderSend failed signal=dbd85daa... broker=fd57d46f... Leg=1/1 op=BuyStop price=4138: MTAPI OrderSendSafe timed out**.

Fix for (a) — confirm unknown orders (commit 9fff2fb5, 3 files). When an order send times out, the worker now **looks the order up at the broker**: if it exists, the position is adopted and recorded; if it provably does not exist, the order is re-sent. That turns the original "outcome unknown" alert into something that can be resolved. Eight tests cover it, and it is on by default. Supporting change in 931347e9 : the timeout is now configurable and must stay below the bridge's 60-second limit, and a timed-out order can be re-sent safely under a separate switch (off by default).

b) The rate-limit storm. The broker allows us a certain number of requests per second and refuses everything once we go over it. Because of the loops described below, we went over it constantly: in under two hours that evening, **350 requests were refused and 36 different customer accounts were affected**. Those customers had done nothing wrong — their settings were fine — they simply queued behind our own noise and were turned away. Staging, which never ran the looping jobs, saw **zero** refusals all day, which is how we know the two environments differed because of our own traffic and not because of the broker.

Evidence from the production logs, 20:00–21:55 on 5 October:

Measure	Production	Staging
"Too many requests" / rate-limit lines	350, across 36 customer accounts	0 all day
Error events (whole day)	3,347	47
Distinct error types (whole day)	43 issues / 3,394 events combined	

Busiest error types that day, to show this was not one isolated fault: unreconciled open trades 2,835 · positions recorded in the app but not at the broker 255 · no matching trading channel 95 · rate limited 15 · "too many requests" 8 · timeout 1.

The errors as they appeared:

- Railway, through the evening: 20:00:00 [autoManagementMonitor] /Quote failed for XAUUSD.s (account=6ca962e0...): Too many requests, 20:01:07 [tradeBrokerDriftMonitor] OpenedOrders failed broker=44ca567b...: Too many requests, 20:01:07 [signalEntryPendingMonitor] /OpenedOrders failed account=83efa384...: Too many requests** — the same three words, line after line, for two hours.
- Railway, the bridge saturated at 17:11: 17:11:14 [openTradeReconcileMonitor] reconcile failed broker=7399156c...: MTAPI OpenedOrders timed out, 17:11:16 [tradeExecutor] /SymbolParams failed uuid=a93f5ec4... symbol=XAUUSD: MTAPI SymbolParams timed out, 17:11:13 [closedTradeFillMonitor] history read failed broker=d658debc...: MTAPI OrderHistoryPagination timed out** — three different jobs, within seconds of each other, every call against the bridge timing out.
- Sentry: rate limited** (15 events), **"too many requests"** (8), **timeout** (1) — the names Sentry recorded for the refusals that day.

Fix for (b) — spread the load (commit 931347e9). Requests are now **paced evenly across both bridge channels** instead of arriving in bursts. The broker answers the health check with a short quoted text value, "OK" , instead of a full message. Our code only treated replies that begin with { or [as JSON, so that answer was never unwrapped — and the check then compared the text together with its quotation marks, found no match, and recorded a **successful** reply as a failure on every account. The worker now parses replies by their content type whatever their shape, so the health check tells the truth. That matters because the clean-up job described under root cause 3 accepts an empty account as proof only when this check passes — with it permanently failing, that job deferred every time and stale records were never closed. Session clean-up is bounded so it cannot run away. On top of that, the waiting periods described under (c) cut the total number of requests, which is the main reason the storm cannot rebuild.

c) The two loops that never stopped. Two jobs in the worker each keep a list of work to do. Normally the list empties as each job is finished. On 5 October both lists held items that could never succeed — a position the broker no longer had, a task trying to close a slice of a position that had already been closed — and nothing ever removed them. So the jobs kept picking up the same impossible items and trying again, with no waiting period and no attempt limit, forever. Every attempt was a real request to the broker, which is exactly what pushed us over the limit described above.

Job that applies stop-loss settings: 1,108 failed attempts in 5 minutes from just 15 records (15:27-15:29, continuous)
Job that handles partial take-profits: 1 failed log entry every ~7 seconds from a single task - 571 entries in 70 minutes

Read plainly: 15 stuck records produced over a thousand failed attempts in five minutes, and one stuck task produced more than eight attempts a minute for over an hour. Neither job ever reached a conclusion; both simply carried on until we stopped them.

The errors as they appeared:

- **Sentry: reconciliation_failed: Open trade reconciliation failed for broker account** — the busiest error of the day — and **unmanaged_live_position: Broker holds a live position with no trade row for this user** (250 events).
- **Railway: 15:27:00 [autoManagementMonitor] apply failed trade=d96c0580... ticket=402106270: automatic management reconciliation required: stored ticket has no live position match** — 1,108 of these in five minutes — and **[partialTpMonitor] fire failed partial=... ticket=: partial close reconciliation required: stored ticket has no live position match** , the stuck task's own message, 83 times in two hours.
- **Railway, the clean-up giving up: 17:11:15 [openTradeReconcile] empty OpenedOrders with 1 tracked open trade(s) account=e72773cd... – deferring ghost close (suspected disconnect) .**

Fix for (c) — wait, quarantine, park (commits 931347e9 and bd751498).

- The stop-loss job now fetches the account's open positions **once per account per attempt** instead of once per record, waits longer after each failure, quarantines records that keep failing, and parks them entirely after repeated failure.
- The partial take-profit job **waits before contacting the broker at all**: unfixable errors 30 seconds doubling to 30 minutes, ordinary hiccups such as rate limits 5 to 60 seconds, and parks for six hours after twelve unfixable attempts. It writes **at most one failure record per task per five minutes** instead of one per attempt.
- A stuck task can only be **closed off after four pieces of evidence, all of them**: enough failed attempts; a fresh, healthy connection check; the position absent from **two separate** readings of the account's live positions; and a genuine closing entry in the broker's own history for that exact ticket. The history test uses the rules of the account's own broker connection, and a wrong connection type answers "no proof", never "proof". The main trade record is never modified by this path.
- A failed price request is counted separately, so a flaky price feed cannot reset a task's failure count or delay healthy tasks.
- A successful close is recorded safely: the update only applies while the record is still in the state we expect and is retried, so a slow database write can never leave an already-completed close pending and cause the same slice to be closed twice.
- Recording a trade as closed at the broker now additionally requires matching history evidence, so stale records stop feeding the loop.

d) The event table. Every attempt — successful or not — writes a row to the event-log table, and nothing ever deleted from that table. So the table grew and grew until it reached **4.42 GB**, leaving the database close to its size limit and at risk of refusing further writes. It also made this incident hard to read: one stuck task alone added 571 rows in 70 minutes, burying the few lines that actually mattered under thousands that said the same thing.

Fix for (d) — daily pruning (SQL applied to production on 5 October). A retention policy now removes old event-log rows daily and the backlog was purged. **4.42 GB → 477 MB.**

3. Affected user(s)

User	Signal	Symbol	Lot	Opened (UTC)	Record in our database
7833f81b...	dbd85daa...	XAUUSD (pending buy)	0.06	2026-10-05 15:28:02	signal marked failed; no trade row
081d5976... (Xlence-Real1, login 5510044400)	cbf88eeb...	XAUUSD (buy)	0.02 ×3	2026-10-05 17:12:38	1 trade row (eab120e6 , still marked open) + 2 positions with no record at all
36 customer accounts	—	mixed	—	20:00–21:55	refused with "too many requests"

Broker account fd57d46f... — MT5 demo, login 10012882280 , connected (the unconfirmed pending order).

Proof A — the unconfirmed order is no longer open (account is flat)

Read directly from the broker for account 081d5976... : the connection check returned 200 "OK" , and the list of open positions returned **none**.

ticket 12572015	Buy 0.02 XAUUSD @ 4126.46 opened 17:12:38.306Z closed 17:29:50.987Z @ 4129.67 volume closed 0.02 profit +6.42 close comment "" (closed by hand, not by stop-loss or take-profit) ref TScopier:INVESTMENTVI:cbf88eeb
-----------------	--

Our database still shows that row as **open** — so it is a **stale record**, exactly the kind the clean-up job in root cause 3 exists to remove. It is still there because production runs the old code: the health check fails, the job defers, and the row is never corrected. There are no worker log entries between 17:15 and 17:30, so the close was almost certainly done by hand in the trading terminal.

Proof B — one signal opened three positions, only one was recorded

Signal cbf88eeb ("GOLD BUY 4126 / TP1 4129 / TP2 4133 / TP3 4137 / SL 4116", lot size not stated → 0.02, one leg expected) produced three positions within one millisecond:

Ticket	Stop-loss	Take-profit	Opened	Closed	Result	Trade record
12572015	4116.00	4137	17:12:38.306Z	17:29:50.987Z	+6.42	yes — eab120e6 (still marked open)
12572016	4120.38	4137	17:12:38.306Z	17:19:56Z	+1.50	MISSING
12572017	4122.05	4137	17:12:38.306Z	17:15:28Z	+7.62	MISSING

The log for that same second shows why:

17:12:00 first attempt to place the order, timed out, retrying (1 of 3)

17:12:21 second attempt, timed out, retrying (2 of 3)

17:12:38 order confirmed by the broker, ticket 12572016, took 58.4 seconds

Two blind retries after our side timed out, then the broker processed all three attempts together. **0.04 lots were running with no trade record for three to seven minutes.** The ticket the broker reported back (12572016) also does not match the ticket we stored (12572015 , whose stop-loss matches 12572017). The cause of this part is still open — see section 8.

4. Root cause

The broker connection was overwhelmed by requests of our own making; the 20-second timeout is a symptom, not the disease.

1. **The stop-loss management job retried forever.** It runs every 400 milliseconds and was working through 15 records that could never be applied — every attempt returned "the stored ticket has no live position match". They never left the work list, there was no waiting period between attempts, no attempt limit, and no way for the job to conclude and move on. Result: 1,108 failed attempts in five minutes, each one a real request.
2. **The partial take-profit job retried forever.** One stuck task kept asking the broker to close a slice of a position that no longer existed — once every tick, writing a failure record every seven seconds (571 in 70 minutes) and consuming the shared request allowance each time.
3. **The clean-up job never caught up.** What does the cleaning is a job inside the trading worker, started alongside the two jobs from (1) and (2). It runs every 30 seconds while there is work, every 120 seconds when idle, and handles up to 500 rows at a time. It takes every trade row our database still shows as open, asks the broker which positions are actually open on that account, and marks a row closed once its position has gone — but only when three things hold together: the account passes its health check, a second reading of open positions is still empty, and the broker's own history contains a closing entry for that exact ticket. Anything less and the row is left alone for review. What it cleans is **stale "open" rows**: rows whose position was closed at the broker (by hand, by stop-loss or take-profit) while our database still shows them as open — for example the row for ticket 12572015 in Proof A, still marked open a day later. Those stale rows are exactly the work items the jobs in (1) and (2) retried forever, so this job is what should have emptied their lists. It failed 113,000+ times since 25 August (2,835 on 5 October alone), and one reason is structural: an empty list of positions is only accepted as proof when the health check passes, and that check reported failure on every account (see fix (b) above), so the job deferred every single time and the stale rows piled up. This job is **not** what caused the rate limits — the loops were — but without it the loops never ran out of work.
4. **Our timeout was shorter than the bridge's.** The worker gave up after 20 seconds while the bridge allows 60 — we quit before the bridge did. Worse, an order that timed out was never afterwards checked with the broker, so the uncertainty was permanent.
5. **The result under load:** a perfectly normal order exceeds 20 seconds → recorded as unknown → two alerts for one event → the copy fails. The same mechanism produced the morning's "too many requests" storm and the 350 rate-limited lines across 36 accounts that evening.

Chain: two work lists that never empty × retries every 400 ms → the bridge is saturated → legitimate orders exceed 20 seconds → failed copies, and every other customer gets refused for rate limiting.

Same day, unrelated cause: the event-log table reached 4.42 GB because no retention policy existed — a data-growth problem, not a loop.

5. Files changed

Commit	Files	What
931347e9	14	order placement, request pacing, stop-loss job, session handling, open-trade reconciliation + tests
9fff2fb5	3	new unknown-order reconciliation (worker/src/tradeExecutor/reconcileUnknownSend.ts), wired into order execution, with tests
bd751498	6	partial take-profit job, its tests, open-trade classification, shared error classification (new), stop-loss job
SQL	1	supabase/migrations/20261005120000_listener_events_retention.sql

6. Verification

Four rounds of code review:

Round	Verdict	Notes
1	Failed	Serious: the proof that a position was gone was a naive ticket scan — it could claim to have looked when it had not
2	Passed with notes	7 medium findings — all fixed
3	Passed	2 medium + 8 low — all addressed, including the safe recording of a successful close and the provider-specific history test
4	Passed	Round-3 fixes confirmed; one further medium issue (a database error path) fixed with a retrying write

Tests:

Check	Result
Type check (full worker build)	clean
Lint on all 6 changed files	clean
Targeted test suites (run one at a time)	89 / 89
Complete worker test suite (216 files, 49 minutes)	2,210 / 2,210 passed, 0 failed
Partial take-profit suite	17 → 28 tests

Tests were run one file at a time: running several at once nearly exhausted the machine's memory.

Statement on customer money. Nothing in these changes can close, cancel or duplicate a position that is still open at the broker. The single call that closes part of a position sits behind a step that refuses to proceed unless the position can be positively identified, and the path that closes off a stuck task never contacts the broker at all — it only updates an internal record after four independent proofs. Any failed or incomplete check results in "do nothing and wait".

Evidence after deployment (staging, 6 October):

06:37:21Z deployment successful - staging branch, commit bd751498 (listener, trade and backtest services)
06:38:09Z partial take-profit job started
06:38:09Z first tick completed, 4 tasks pending

Window 06:37 → 08:00 UTC (83 minutes, 869 log lines): **zero** matches for failed closes, terminal cancels, parking notices, lost claims, "too many requests" or rate-limit errors.

7. Deployment status

Environment	Service	Deployed (UTC)	Branch	Commit	Status
staging	listener, trade, backtest	2026-10-06 06:37:21	staging	bd751498	successful

production	listener, backtest, worker	2026-10-04 18:38:32	main	e188ff09 (PR #158)	successful — contains none of these fixes
------------	----------------------------	---------------------	------	--------------------	---

Pushed to both `origin/staging` and `upstream/staging` as a verified clean fast-forward. The event-log retention policy **is** already applied in production, because that was a database change and does not depend on a code deployment.

Production therefore still runs the old code: the rate-limit storm and the stale "open" records can still occur there until staging is promoted to production.

8. Follow-ups

1. **Promote staging to production** — a separate, explicit decision; not taken here.
2. **The signal that opened three positions** (`cbf88eeb...`) — three positions, one trade record, 0.04 lots untracked for 3–7 minutes, and the reported ticket does not match the stored one. Working notes in `docs/scratchpads/scratchpad-duplicate-fill-cbf88eeb-2026-10-05.md` ; it deserves its own incident report once the mechanism is proven.
3. **Stale "open" trade records** (for example `eab120e6` , ticket `12572015` , now proven closed) — 18 known orphans; cleaning them up needs a reviewed, evidence-gated change.
4. **Confirm the pending order** on login `10012882280` and adopt or clear it.
5. **Fix unreconciled open trades** (2,835 a day) — this is what keeps both jobs supplied with impossible work.
6. **Merge the duplicate alerts** for a single timeout into one.
7. **History pagination** — the proof reads one page of broker history, so on a busy account a closing entry on page two would postpone the decision indefinitely. This is the safe direction (nothing is ever cancelled wrongly), but the cancel half may never run; the existing paginated reader should be reused.
8. **Keep the job audible while it waits** (it currently goes quiet during long waits) and **park tasks that fail persistently for ordinary reasons** such as the market being closed.
9. **Store wait-times centrally** so all replicas share them, and document the new settings in the environment template.
10. Rotate the secrets that were pasted into the chat during diagnosis (database service key, broker keys, credential encryption key, AI and Telegram keys) — still outstanding.